

AskAnyDoc RAG Architecture

Compact guide to multi-format ingestion, provenance, retrieval, and incremental delivery

The central decision

AskAnyDoc v1 accepts **PDF, DOCX, TXT, and Markdown**. File format changes only the extraction step. After extraction, every source becomes common text plus provenance and enters the same chunking, Titan embedding, and Aurora PostgreSQL/pgvector pipeline.

```
PDF / DOCX / TXT / MD -> S3 -> Ingestion Lambda -> matching extractor
-> normalized text + provenance -> chunks -> Titan -> Aurora + pgvector
```

Version 1 boundary

Format	Status	Primary citation location
PDF (.pdf)	Supported	Physical PDF page
Word (.docx)	Supported	Heading or section; page only if reliable
Plain text (.txt)	Supported	Line range and chunk position
Markdown (.md)	Supported	Heading or section path
Excel / CSV	Later	Needs structure-aware extraction
PowerPoint	Later	Slide-aware extraction
Images / scanned OCR	Later	OCR or document understanding

Why DOCX is not page-first

Word displays pages after layout is calculated from fonts, margins, and rendering settings. Native DOCX structure reliably provides headings, sections, paragraphs, and tables. Cite those first; retain a page only when an external rendering process makes it dependable.

Durable rule: preserve the original file, normalize early, and keep provenance attached through every later step.

Ingestion orchestration

One S3-triggered Lambda is the conductor. It decodes the event, validates the file type, selects an extractor, and then coordinates the shared downstream work.

```
S3 ObjectCreated event under uploads/  
|  
v  
INGESTION LAMBDA  
decode bucket + key  
allowlist extension  
select extractor  
|  
+-- .pdf -> PDF extractor  
+-- .docx -> DOCX extractor  
+-- .txt -> TXT extractor  
+-- .md -> Markdown extractor  
+-- other -> ignore + log  
|  
v  
normalized records -> chunk -> Titan -> Aurora/pgvector
```

What each specialist does

Component	Responsibility
S3	Stores originals and emits upload events
Ingestion Lambda	Validates type and coordinates processing
Extractor	Returns clean text plus source location
Chunker	Splits text and preserves provenance
Titan Embeddings	Turns text into vectors
Aurora PostgreSQL	Stores text, vector, and metadata together
pgvector	Performs similarity search

Important boundary

The document itself is not sent to Titan. The selected extractor first turns it into text. Titan receives chunk text and returns an embedding vector to Lambda.

Common records and provenance

Every extractor returns the same outer record shape. The **location** object is the only format-specific portion.

Required field	Meaning
document_id	Stable source identity for replacement and deduplication
source_name	Human-readable filename used in citations
source_type	pdf, docx, txt, or md
source_uri	Original S3 URI
object_key	Exact S3 object key
text	Extracted content for shared chunking
location	Page, section, heading path, or line range
chunk_id	Stable ID added during chunking

Format-specific location

```
PDF {"page_number": 12}
DOCX {"heading": "Annual Leave", "section_index": 7}
MD {"heading": "Authentication", "heading_path": ["Architecture", "Authentication"]}
TXT {"line_start": 120, "line_end": 145}
```

Provenance must survive

Extraction -> chunking -> embedding -> database storage -> retrieval -> citation. Never let chunking discard the source fields. Build citations from the metadata returned by Aurora, not from names or locations invented by Claude.

Example citations

PDF: Security Policy.pdf, page 12
DOCX: Employee Handbook.docx, section "Annual Leave"
Markdown: architecture.md, heading "Authentication"
TXT: meeting-notes.txt, lines 120-145

Question flow and delivery sequence

Question -> Answer Lambda -> Titan question vector -> pgvector similarity search
-> Aurora returns chunk text + provenance -> Lambda packages evidence
-> Claude writes grounded answer -> answer + citations return to user

The vector database performs the comparison

Lambda asks for the nearest stored vectors. pgvector performs the similarity search. Aurora returns the matched rows containing readable chunk text and provenance. Lambda packages those rows as evidence; Claude does not search the database itself.

Incremental implementation

Milestone	Proof
1 Event + routing	S3 invokes Lambda; four extensions route; other files are ignored
2 Extractors	PDF, DOCX, TXT, and MD each produce the common record contract
3 Shared RAG path	Normalized records chunk, embed with Titan, and store in Aurora/pgvector
4 Query path	Question retrieves traceable evidence and Claude answers from it

Current milestone

The project now implements milestone 1: one S3 notification under **uploads/**, case-insensitive supported-file detection, URL-decoded keys, stable document identity, named extractor routing, and safe rejection of unsupported formats. Extractor calls remain deliberately unconnected until the event boundary is proven in AWS.

Thirty-second refresher

Lambda coordinates; extractors read formats; Titan embeds; Aurora stores; pgvector searches; Claude explains. File format matters during extraction. Everything after normalized text plus provenance is shared.